

ATmega32A Module

SPI (Serial Peripheral Interface)

Ali Sahafi

1 Theory

SPI is a fast, synchronous bus for talking to peripherals such as displays, SD cards, sensors and shift registers. One **master** generates the clock; one or more **slaves** respond. Four wires:

Signal	Pin	Direction (master)	Meaning
MOSI	PB5	output	Master Out, Slave In
MISO	PB6	input	Master In, Slave Out
SCK	PB7	output	Clock (generated by master)
SS	PB4	output	Slave Select (active low)

SPI is built around a **shift register**: with every clock pulse one bit goes out on MOSI *and* one bit comes in on MISO. Sending and receiving are the same operation — **transfer()** always returns the byte the slave shifted back. To “only read”, you send a dummy byte.

Modes. Clock polarity (CPOL) and phase (CPHA) give four modes (`SPI_MODE_0` ... `SPI_MODE_3`); the datasheet of the slave device tells you which one to use. Mode 0 is by far the most common.

2 Registers

Register	Role	Bits used by the driver
SPCR	Control	SPE, MSTR, CPOL, CPHA, DORD, SPR1:0, SPIE
SPSR	Status	SPIF (transfer complete), SPI2X (double speed)
SPDR	Data	write to start a transfer, read to get the answer

3 API reference

Method	Description
<code>SPI.initMaster(mode, prescaler, order, irq)</code>	Configure as SPI master
<code>SPI.initSlave(mode, order, irq)</code>	Configure as SPI slave
<code>SPI.transfer(data)</code>	Synchronous transfer (returns received byte)
<code>SPI.attachInterrupt(callback)</code>	Transfer-complete callback: <code>void cb(uint8_t)</code>

- **Mode:** `SPI_MODE_0/1/2/3`
- **Prescaler:** `SPI_PRESCALER_4/16/64/128`
- **Double-speed prescaler:** `SPI_PRESCALER_2_2X / 8_2X / 32_2X / 64_2X`
- **Data order:** `SPI_MSB_FIRST, SPI_LSB_FIRST`

Pins are configured automatically. `initMaster()` and `initSlave()` set the DDR bits for PB4–PB7 correctly; you do not touch `DDRB` yourself. As master, keep PB4 (SS) an output — if it floats and reads LOW, the hardware silently drops out of master mode.

4 Example: loopback test

The simplest possible SPI experiment needs no slave at all: connect **MOSI to MISO with one jumper wire** and every transmitted byte comes straight back.

Wiring.

- Jumper wire from **PB5 (MOSI)** to **PB6 (MISO)**.
- UART on PD0/PD1 at 9600 baud to watch the results (Module 6).

Build & flash. `make spi`

```
/*
 * SPI Example -- master sends a counter, prints what comes back over UART
 *
 * Simplest test: connect MOSI to MISO with a jumper wire (loopback) and
 * every byte sent comes straight back. With a real slave device, the
 * returned byte is whatever the slave shifted out.
 *
 * Wiring:
 * - PB5 (MOSI) -> slave MOSI (or jumper to PB6 for loopback)
 * - PB6 (MISO) <- slave MISO
 * - PB7 (SCK) -> slave SCK
 * - PB4 (SS) -> slave SS
 * - UART on PD0/PD1 at 9600 baud to see the results
 *
 * Build & flash: make spi
 */

#define F_CPU 8000000UL
#include "../uart/uart.hpp"
#include "spi.hpp"

int main() {
    UART.init(9600);
    SPI.initMaster(SPI_MODE_0, SPI_PRESCALER_16); // pins configured automatically

    UART.print("SPI master ready (loopback test: jumper MOSI-MISO)\r\n");

    uint8_t counter = 0;
    while (true) {
        uint8_t received = SPI.transfer(counter);

        UART.print("Sent: ");
        UART.print((uint16_t)counter);
        UART.print(" Received: ");
        UART.print((uint16_t)received);
        UART.print(received == counter ? " OK\r\n" : " MISMATCH\r\n");

        counter++;
        _delay_ms(500);
    }
}
```

```
return 0;  
}
```

Remove the jumper and every read returns a constant — there is nothing driving MISO anymore.

5 Exercises

1. Program a second board with `SPI.initSlave()` and an interrupt callback that stores the received byte; wire the two boards together and exchange a counter.
2. Connect a 74HC595 shift register ($SER \leftarrow MOSI$, $SRCLK \leftarrow SCK$, $RCLK \leftarrow SS$) and drive 8 LEDs over SPI — three wires instead of eight.
3. Change the data order to `SPI_LSB_FIRST` on the master only. What arrives at the slave when you send `0x01`? Verify by experiment.